MCPulse - Demonstration Guide

For us (the team), to rehearse and then present the project to the professor. Everything below was verified on the live deployment on 2026-10-22.

> **Total demo time: 12-15 minutes.** Leave the browser and the AWS console open in two windows before you start. Live URL: `http://54.161.184.121` (the public IP changes whenever the instance is stopped and started - check `aws ec2 describe-instances` or the last line of `scripts/aws/resume.sh`).

---

0. Before the demo (5 minutes)

```
aws-lab-login                    # only if the lab session was restarted
scripts/aws/resume.sh            # start RDS -> wait -> start EC2 -> verify
scripts/aws/allowlist.sh         # refresh the administrator address
```

Then confirm:

```
curl -s -o /dev/null -w '%{http_code}\n' http://54.161.184.121/health/ready   # 200
```

Two browser tabs, side by side: - Tab A -> `http://54.161.184.121` (the dashboard) - Tab B -> the AWS console, already signed in, on the **EC2 -> Instances** page

One terminal window with these ready (they are read-only, safe to run live):

```
export AWS_PROFILE=learnerlab
aws sts get-caller-identity --query Account --output text
aws ec2 describe-instances --filters Name=tag:Name,Values=mcpulse-server \
  --query "Reservations[].Instances[].{id:InstanceId,state:State.Name,type:InstanceType,ip:PublicIpAddress}" --
aws rds describe-db-instances --db-instance-identifier mcpulse-db \
  --query "DBInstances[0].{id:DBInstanceIdentifier,status:DBInstanceStatus,public:PubliclyAccessible,class:DBIn
```

---

1. The opening (1 min) - what the problem is

> "MCP servers are how AI agents call tools, but they are black boxes. If one breaks, an agent just fails - there is no record of *when* it broke, *how often*, or *why*. MCPulse is a monitoring platform for MCP servers: it probes them with the real protocol handshake, measures latency, stores every observation, opens incidents when they fail and resolves them on recovery. It is deployed on AWS and I will show you the running deployment, the data flow and the evidence."

Say this while showing the dashboard (Tab A). Point at: the status counts, the five providers, the latency column, the incidents list.

---

2. The product working (4 min)

2.1 Live monitoring - Tab A

- The **Latency** column and the **Last Check** timestamp refresh every 60 seconds. Say: *"This is not a static page - the API checks the endpoints and the dashboard shows the real round-trip time."*
- Point out the statuses: **Hugging Face** and **Google Drive** are `Healthy` (HTTP 200). **GitHub**, **Cloudflare** and **Brave Search** are `Down` because they require a token (HTTP 401 / 403).
- Say: *"That difference is the point of the product. MCPulse does not pretend an endpoint is fine - it records the exact HTTP status and explains it."*

2.2 Tool discovery - Tab A (this is the click that used to fail)

1. Click **Discover Tools** on **Hugging Face Hub**.
2. The tool catalogue appears: `hf_fs`, `hf_whoami`, `hub_repo_details`, `hub_repo_search`, each with its description and JSON Schema.
3. Say: *"This is a live `tools/list` call over the MCP protocol against a public server, and the schemas are persisted in PostgreSQL."*

> If the button returns 401: the administrative key is missing from the browser. Click **Set admin key** in the header and paste `cat ~/.mcpulse/admin-key`. The dashboard now opens the editor by itself when a key is missing, so it can never fail silently.

2.3 The incident lifecycle - Tab A + terminal

The chaos simulator was removed on request, so demonstrate the incident machine with a **real** failing endpoint:

```
# 1. register an unreachable endpoint (public read, needs no key)
curl -s -X POST http://54.161.184.121/api/servers \
  -H 'Content-Type: application/json' \
  -H "X-MCPulse-Admin-Key: $(cat ~/.mcpulse/admin-key)" \
  -d '{"name":"Demo outage probe","endpoint":"https://unreachable.invalid/mcp",
       "transportType":"streamable_http","enabled":true,"checkIntervalSeconds":30}'

# 2. wait ~90 s for three consecutive failures, then show the dashboard:
#    status = Down, an open incident appears

# 3. delete the probe again
```

What to say: *"Three consecutive failures open an incident. The first successful check afterwards closes it automatically. The threshold, the incident table and the resolution are all in the code - not manual."*

Alternative if there is no time: show the existing incident rows in the dashboard's incident list and in the database:

```
aws rds ... # or use the dashboard's Incidents view
```

2.4 Telemetry export - Tab A

Click **Export CSV** in the header. Say: *"Requirement FR-12: the full check history - timestamp, duration, outcome, HTTP status, sanitised error - exported as a downloadable report, generated directly from the database."*

2.5 The evidence-based diagnostics

Open a provider row -> the AI diagnostics card. Say: *"It separates measured facts from hypotheses. For GitHub it will tell you the endpoint requires a bearer token with the right scope - that is a hypothesis derived from the recorded 401, not a guess."*

---

3. The AWS deployment (5 min) - this is the part for the poster

3.1 The services (Tab B)

Walk the console in this order, one sentence each:

Console page	One-liner
EC2 -> Instances	*"The application tier. One t3.small Amazon Linux instance runs nginx and the Node API in Docker."*
EC2 -> Security Groups (web)	*"Port 80 and 443 open for the dashboard, SSH from one address only. The API port 3001 is deliberately not exposed."*
EC2 -> Security Groups (db)	*"PostgreSQL 5432 is accepted only from the web security group."*
RDS -> Databases	*"Private PostgreSQL 16 instance, 20 GB gp3, no public access. The endpoint the API connects to over TLS."*
CloudWatch -> Log groups	*"`/mcpulse/…` - application and host logs with 7-day retention."*
CloudWatch -> Metrics -> CWAgent	*"Memory and disk utilisation of the instance."*

3.2 The data flow (draw this on the board / show the diagram)

```
Browser   80    nginx (EC2)     Express API   TLS 5432    RDS PostgreSQL

                                        HTTPS 443    the 5 monitored MCP providers

                              CloudWatch (logs + metrics)
```

> "One inbound path, one outbound path, one database. The browser never talks to the database; the database is never exposed; the API is the only thing that talks to the outside world - and that is exactly where the SSRF guard and the TLS verification sit."

3.3 Prove the isolation (terminal, 20 seconds)

```
# the database has no public address
aws rds describe-db-instances --db-instance-identifier mcpulse-db \
  --query "DBInstances[0].PubliclyAccessible" --output text      # -> False

# and the API port is loopback only
ssh -i ~/.mcpulse/mcpulse-lab.pem ec2-user@54.161.184.121 \
  'sudo docker compose -f ~/mcpulse/compose.aws.yaml ps'          # -> 127.0.0.1:3001->3001/tcp
```

3.4 Prove the data really is in the database

```
aws rds ... # describe-db-instances -> db.t3.micro, 20 GB gp3, Private
```

Or, on the instance:

```
sudo docker exec mcpulse-aws-api-1 node -e "...SELECT count(*) FROM monitoring_checks..."
```

Say: *"538 checks, 5 incidents, 4 tools, 3 applied migrations - the numbers come from the RDS instance, not from a JSON file."*

---

4. The engineering story (2 min) - the questions that come up

Have these ready; they are the questions a professor asks.

Question	Answer
Why PostgreSQL on RDS and not a file?	Time-series queries over checks, incidents and tools with relations and a transactional migration engine. RDS also gives patching, durability and isolation for free.
Why is the database private?	`PubliclyAccessible = false` plus a security group that only accepts 5432 from the web tier. RDS refuses unencrypted connections, so the link is TLS with the RDS CA bundle and the hostname verified.
How do you know the checks are real?	Each check is a real MCP `initialize` JSON-RPC request with protocol version 2025-03-26, and the measured round-trip time is stored per check. 538 rows.
How are the "Down" providers handled?	They need a token. MCPulse records the actual HTTP status (401/403) and the diagnostics report names the cause. It does not mask the failure.
What prevents SSRF?	A guard that rejects loopback, private, link-local and metadata destinations, applied before every outbound request. Registering `http://169.254.169.254/…` returns 400.
How is the app secured?	The API port is bound to loopback; nginx is the only public entry point; state-changing requests require an administrative key; SSH is restricted to one address.
What did the lab limit?	`iam:PassRole` is denied, so no instance profile can be attached - the CloudWatch agent uses the temporary credentials instead. A custom DB subnet group is also rejected. Both are documented.
How do you reproduce it?	`scripts/aws/deploy.sh` is idempotent; `resume.sh` restarts it after a lab session; `stop.sh` stops the resources without deleting anything.
What does it cost?	≈ $0.18 for a four-hour session: EC2 $0.021/h, RDS $0.017/h, 20 GB gp3, public IPv4. No load balancer, no NAT gateway, no S3.
What would you add next?	OAuth/PAT support per provider, SLO and burn-rate alerts, X-Ray tracing, automated remediation, and a Terraform module.

---

5. Fallbacks if something breaks

Symptom	Fix
The URL does not load	The IP changed -> `scripts/aws/allowlist.sh` prints the current one.
The dashboard loads but a button returns 401	Click **Set admin key** and paste `cat ~/.mcpulse/admin-key`.
The campus network blocks the page	Use the browser, not `curl`; the institutional filter blocks raw-IP HTTP.
The instance is stopped	`scripts/aws/resume.sh` (starts RDS first, then EC2).
Credentials expired	`aws-lab-login`, then `scripts/aws/resume.sh`.
No CloudWatch log events	Known limitation in this account - the groups and the metric definitions exist and the agent runs; show the database evidence instead and explain the cause.
Port 22 unreachable after a network change	`scripts/aws/allowlist.sh` refreshes the security group rule.

---

6. The closing (30 s)

> "MCPulse turns an opaque AI integration into an observable service: every check is measured and stored, every failure becomes a tracked incident, and the whole platform runs on AWS with the database isolated, the traffic encrypted and the cost under twenty cents for a lab session. The code, the automation scripts and the deployment evidence are in the repository."

Then: *"Happy to take questions."*

---

7. Cheat sheet - the numbers to quote

```
deployment        EC2 i-0059c1813dd3d1ada (t3.small, AL2023) - RDS mcpulse-db (PostgreSQL 16.13, db.t3.micro, 2
database rows     5 servers - 538 checks - 5 incidents - 4 tools - 3 migrations
providers         GitHub - Google Drive - Brave Search - Cloudflare - Hugging Face
live latency      Hugging Face ~35 ms - Google Drive ~38 ms - GitHub 401 - Cloudflare 401 - Brave 403
tools discovered  hf_fs - hf_whoami - hub_repo_details - hub_repo_search
security          public access = false - TLS verified - API loopback-only - admin key required for writes - SS
cost              ≈ $0.18 / 4-hour session
scripts           audit - deploy - resume - stop - allowlist - cloudwatch
```